

# DBMS Skill Task: Rating & Analytics System

|                           |                                    |                                     |
|---------------------------|------------------------------------|-------------------------------------|
| Course: DBMS Lab (Week 6) | Brand Domain: One8 Shoe E-Commerce | Module: Numerical Ratings & Metrics |
|---------------------------|------------------------------------|-------------------------------------|

## 1. System Context & Analytics Purpose

The Rating Management System processes quantitative user ratings (1.0 to 5.0 scale) for One8 footwear. Using SQL aggregate functions like `AVG()`, `COUNT()`, and `MAX()`, the engine computes real-time rating summaries and isolates high-performing products.

## 2. Relational Schema Architecture

| Column Name  | Data Type    | Key / Constraint                | Description                          |
|--------------|--------------|---------------------------------|--------------------------------------|
| rating_id    | INT          | PRIMARY KEY, AUTO_INC           | Unique rating record identifier.     |
| product_id   | INT          | NOT NULL, FOREIGN KEY           | Target shoe product ID.              |
| customer_id  | INT          | NOT NULL                        | Submitting customer ID.              |
| rating_score | DECIMAL(2,1) | CHECK (1.0 TO 5.0)              | Numerical rating score.              |
| --           | --           | UNIQUE(product_id, customer_id) | Prevents duplicate ratings per user. |

## 3. Table Definition & Constraint Implementation

```
CREATE TABLE product_ratings ( rating_id INT PRIMARY KEY AUTO_INCREMENT, product_id INT NOT NULL, customer_id INT NOT NULL, rating_score DECIMAL(2, 1) NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, CONSTRAINT chk_rating_range CHECK (rating_score BETWEEN 1.0 AND 5.0), CONSTRAINT unq_customer_product UNIQUE (product_id, customer_id) );
```

1.0 – 5.0  
Allowed Range

UNIQUE  
User-Product Rule

DECIMAL(2,1)  
Precision Format

## 4. Aggregation Query & Calculations

Calculating overall mean score, total rating count, and spread per product using SQL aggregate functions:

```
SELECT product_id, COUNT(rating_id) AS total_ratings, ROUND(AVG(rating_score), 2) AS
average_rating, MAX(rating_score) AS highest_rating, MIN(rating_score) AS lowest_rating FROM
product_ratings GROUP BY product_id ORDER BY average_rating DESC;
```

| Product ID | Product Name (Reference) | Total Ratings | Average Rating | Max Rating | Min Rating |
|------------|--------------------------|---------------|----------------|------------|------------|
| 101        | One8 Fuse 2.0            | 4             | 4.83           | 5.0        | 4.5        |
| 103        | One8 Classic Leather     | 3             | 4.90           | 5.0        | 4.8        |
| 102        | One8 Drift Cat           | 3             | 3.77           | 4.0        | 3.5        |
| 104        | One8 Training Lite       | 2             | 2.75           | 3.0        | 2.5        |

## 5. Identifying Highly Rated Products (HAVING Clause)

Filtering items that meet the platform's high-rating threshold (Average Rating ≥ 4.5):

```
SELECT product_id, COUNT(rating_id) AS total_reviews, ROUND(AVG(rating_score), 2) AS
avg_rating FROM product_ratings GROUP BY product_id HAVING AVG(rating_score) >= 4.5 ORDER BY
avg_rating DESC;
```

| Product ID | Total Reviews | Calculated Average Score | Status Badge           |
|------------|---------------|--------------------------|------------------------|
| 103        | 3             | 4.90 / 5.0               | Top Rated / Trending   |
| 101        | 4             | 4.83 / 5.0               | Top Rated / Bestseller |

**Key takeaway:** Products 101 and 103 qualify for the "Top Rated" promotional section on the One8 storefront, whereas products 102 and 104 are filtered out automatically by the HAVING clause.